

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning

## 1 Dataset Source

- Dataset Used: Iris Dataset
- Source Link:
  - 👉 <https://www.kaggle.com/datasets/uciml/iris>
  - 👉 (Direct CSV used in code):  
<https://raw.githubusercontent.com/uiuc-cse/data-fa14/gh-pages/data/iris.csv>

## 2 Dataset Description

The Iris dataset is a well-known dataset used for classification tasks.

- **Total Samples:** 150
- **Number of Features:** 4
- **Target Variable:** Species
- ♦ **Features:**
  1. Sepal Length (cm)
  2. Sepal Width (cm)
  3. Petal Length (cm)
  4. Petal Width (cm)
- ♦ **Target Classes:**
  - Setosa
  - Versicolor
  - Virginica
- ♦ **Characteristics:**
  - Balanced dataset (50 samples per class)
  - Numerical features
  - Linearly separable (especially Setosa)
  - Suitable for testing classification algorithms

## 3 Mathematical Formulation of SVM

Support Vector Machine tries to find the optimal hyperplane that separates classes.

### ♦ Linear SVM Equation:

$$w \cdot x + b = 0$$

Where:

- $w$  = weight vector
- $x$  = input features
- $b$  = bias

### ♦ Optimization Objective:

$$\min \frac{1}{2} \|w\|^2 + C \sum \xi_i \quad \min \frac{1}{2} \|w\|^2 + C \sum \xi_i$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

Where:

- $C$  = regularization parameter
- $\xi_i$  = slack variable

### ♦ Kernel Trick:

Used to transform data into higher dimensions:

- Linear Kernel
- Polynomial Kernel
- RBF Kernel

## 4 Algorithm Limitations

- Not suitable for **very large datasets** (high computation time)
- Sensitive to **choice of kernel and parameters**
- Performs poorly when:
  - Data is highly noisy
  - Classes overlap significantly
- Requires **feature scaling**
- Not ideal for **multi-class problems** (handled using strategies like One-vs-One)

## 5 Methodology / Workflow

### ♦ Steps Followed:

1. **Data Collection**
  - Loaded dataset from CSV link
2. **Data Preprocessing**
  - Separated features and target
  - Converted categorical labels into numeric
  - Applied feature scaling (StandardScaler)
3. **Train-Test Split**
  - 80% training, 20% testing
4. **Model Training**
  - Applied Support Vector Machine (SVM)
5. **Hyperparameter Tuning**
  - Used GridSearchCV to find best parameters
6. **Prediction**
  - Predicted test data results
7. **Evaluation**
  - Accuracy
  - Confusion Matrix
  - Classification Report

### ♦ Workflow Diagram (You can draw this in file)

Dataset → Preprocessing → Train-Test Split → Scaling → SVM Model  
→ Hyperparameter Tuning → Prediction → Evaluation

## 6 Performance Analysis

### ♦ Results Obtained:

- **Best Parameters:**
  - C = 10
  - Kernel = Linear
  - Gamma = scale
- **Accuracy:** 96.67%

### ◆ **Confusion Matrix:**

```
[[10 0 0]  
 [ 0 8 1]  
 [ 0 0 11]]
```

### ◆ **Interpretation:**

- Setosa: 100% correct
- Versicolor: 1 misclassification
- Virginica: 100% correct

### ◆ **Classification Metrics:**

- Precision: High for all classes
- Recall: Slightly lower for Versicolor
- F1-score: Balanced performance

### 👉 **Conclusion:**

The model performs very well with only one error, indicating strong classification capability.

## **7 Hyperparameter Tuning**

Hyperparameter tuning was performed using **GridSearchCV**.

### ◆ **Parameters Tuned:**

- **C:** [0.1, 1, 10, 100]
- **Kernel:** ['linear', 'rbf', 'poly']
- **Gamma:** ['scale', 'auto']

### ◆ **Best Parameters Found:**

- C = 10
- Kernel = Linear
- Gamma = scale

### ◆ **Impact on Performance:**

- Improved accuracy from basic model
- Helped select best kernel (linear)
- Reduced misclassification
- Ensured optimal decision boundary

## **Conclusion**

The SVM model with hyperparameter tuning achieved high accuracy (~97%) on the Iris dataset. The linear kernel was found to be most suitable, indicating that the dataset is nearly linearly separable. The model demonstrated excellent performance with minimal error, making SVM a reliable algorithm for classification tasks.